

S1 Text. MI-Workbench: system details

This document collects implementation details of MI-Workbench that are summarized in the main text. The repository README documents every run-configuration key; the module docstrings referred to below are the authoritative statements of behavior.

1. Data model and run lifecycle

All data are validated with Pydantic models. **RunState** tracks the lifecycle, iterations, token totals and stop reason of a run; **IterationResult** records one step (role, status, provider, model, reasoning-effort setting, CLI version, input, cached-input and output tokens, duration, grade, consensus report, code-execution summary and agent-tool provenance); **ReviewResult** holds parsed critiques; **LoopDefinition** holds the workflow graph. A finite state machine governs runs (pending, running, paused, stopped, completed, failed). A run is marked running when its loop starts; on server start, interrupted runs with persisted iterations become paused and can be resumed. A resumed run replays the compiled plan against the stored iterations, so it continues with the step an uninterrupted run would take next, and it restores the latest executor output, the pending feedback, the stopping-signal history and the consensus gate.

2. Retry and error handling

Adapter errors are categorized from the CLI output (**rate_limit**, **quota**, **auth**, **overloaded**, **timeout**, **network**, **max_turns**, **config**, **empty_output**). Rate-limit, timeout, network and overload errors are retried with exponential backoff, $d_k = d_0 2^k (1 + \epsilon)$ with $\epsilon \sim \mathcal{U}(-0.25, 0.25)$, capped at 30s; authentication, quota, turn-limit and configuration errors fail immediately. Failed calls are never treated as empty successful output, and the tokens of failed attempts are counted. Database writes use a 30-s busy timeout and are retried with exponential backoff when SQLite reports lock contention; an iteration whose progress write still fails is queued and persisted with the next successful write or at the end of the run, so concurrent runs do not lose iterations.

3. Output cleaning

CLI agents often prefix answers with narration (“I will read the file...”, “Let me check...”). Before any parsing, a line filter removes forward-looking first-person narration, tool-call narration, sequencing prefixes (“First, I will...”) and meta-commentary (“The following is...”), while preserving fenced code blocks, Markdown headers and past-tense statements; runs of blank lines left by removals are collapsed. Reviewer outputs are parsed from their JSON critique block, which the filter never touches.

4. Adapter invocations

Adapter	Command (flags only when set)
Claude Code	<code>claude -p -output-format json -max-turns N [-model M] [-effort E] [-system-prompt SP];</code> with tools off: <code>-tools "" -strict-mcp-config</code> and one turn
Codex CLI	<code>codex exec -skip-git-repo-check -ephemeral -json -sandbox read-only workspace-write [-m M] [-c model_reasoning_effort=E] [-cd WS] -;</code> with tools off additionally <code>-c web_search="disabled"</code> , the shell, apps, plugins and multi-agent features disabled, and the user configuration file ignored
Gemini CLI	<code>gemini [-yolo -approval-mode default] [-m M] -o json -p " "</code> with the prompt on standard input; with tools off a system-settings file excludes every built-in tool; <code>@</code> is escaped to prevent file expansion
Mock	in-process, role-aware deterministic responses

5. Knowledge extraction

An optional knowledge-extractor role and a rule-based extractor mine claims from artifacts. Claims are detected by phrase patterns (“results show that...”, “this indicates that...”, metric statements such as “AUROC = X” and p -values) and classified as empirical, methodological, interpretive or comparative. Each claim receives a heuristic uncertainty score $u \in [0.05, 0.95]$ starting at 0.5, lowered by strong evidence (very small p -values), reported confidence intervals and large samples ($n \geq 1000$), and raised by hedging language. For each claim type the extractor proposes falsification tests (stronger baselines for AUROC claims, split-sample validation for layer-specific claims, residualization for attention claims, stricter multiple-testing correction for statistical claims). Claims and their relations (supports, contradicts, extends) are stored in the database and exposed through a graph API. These heuristics are aids for navigating a run’s output; they were not evaluated in this study.

6. Version control and reproducibility packages

Four git modes are supported: none, one commit per run, one commit per iteration, and one branch per run. The reproducibility-package generator writes a ZIP archive with every iteration directory of the run (step outputs, parsed artifacts, consensus records and execution reports), the persisted execution outputs (code, standard output and error, and written files up to a size cap), `run_meta.json`, the resolved prompts, the Python environment, the git state, timing information, and auto-generated dataset card, reproducibility checklist and README.

7. Codebase statistics

Counted on the version described in the paper with the commands in `docs/REPRODUCING.md` (macOS meta-data files and caches excluded).

Metric	Value
Backend source files (excluding tests)	76
Backend lines of code (excluding tests; non-blank)	19,806 (17,157)
Test files (backend)	58
Test functions collected (backend)	1,067
Test lines of code (backend)	16,020
Additional harness tests (benchmark, stopping, case study)	68
Prompt templates	11
Bundled workflow presets	4
API routes	54
Database tables	8
Adapters	4 (Claude Code, Codex, Gemini, mock)